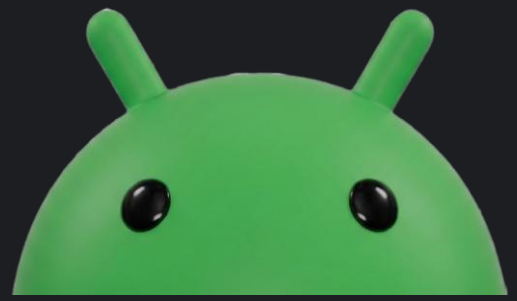
RE:UN10N MONTHLY SHARING SESSION

Intro to Android Security and Mobile Hacking

Lay Back and chit chat together



by w_11

./whoami



w_11

Student, Intern



```
./whoami
CyberX President 25-26

MCC Alumni

RE:UN10N

CAPT

Sleeping Hobbyist
```

Why CAPT? Why Mobile? Why Android?

01 Mobile Hacking Lab might be underrated (Maybe) but its course and Hacking Lab is really good!

02 Mobile apps are everywhere: banking, authenticator, messaging. Also there is tons of pentesting work is mobile too, so it's a skillset that's actually in demand right now.

03 Android holds an estimated 71%+ of mobile OS market share, and being open source makes it a more approachable entry point for newcomers, myself included hehe. But for sure IOS is also one of demanding section for MAPT

What We're Doing Today

I'm skipping most of the OS-internals theory.



- **The toolkit:** Android Studio jadx, apktool, dex2jar, MobSF, quark, Frida, ASC
- **The demo today:** AndroGoat.apk
- **Rooting Android Emulator Using Android Studio**

The One Idea Worth Keeping

If an app keeps a secret, or makes a security decision, on the device itself, then you can do something on it

That's "client-side trust," and it's the most common bug class in mobile:

- **Secrets shipped in the app:** API keys, passwords, or encrypted blobs baked into the APK.
- **Checks that run locally:** root / jailbreak detection, license or PIN checks the app trusts.
- **Weak or home-made crypto:** small key spaces, hardcoded IVs, secrets you can brute force offline.

WORKFLOW

The Loop You'll Repeat

1. Pull — Get the APK off the device or emulator.

2. Static — Read the code and resources without running it.

3. Dynamic — Run it, hook it, watch it, change its behaviour.

4. Prove it — Show real impact, then write it up.

Reading the Code

`jadx / jadx-gui`	Turns the APK's bytecode back into readable Java.
`apktool`	Decodes resources and the manifest, and unpacks to Smali. Also rebuilds for repackaging.
`dex2jar (d2j)`	Converts .dex to a .jar so you can read it in classic Java tools if you prefer them.
`ASC (Droid ASC)`	Handy on huge apps: a fast decompiler that cross-references and pulls single classes in milliseconds. Shared by Mr.Benkyou hahaha, its pretty fast.

Some Extra Tools you will need in pentesting

`MobSF`

Mobile Security Framework. Drop in the APK and it returns a full report: permissions, exported components, hardcoded secrets, insecure settings, and more.

`quark-engine`

Scores an app by behaviour rather than strings, so it still works against obfuscation. Great for spotting what an app actually does.

DYNAMIC TOOLKIT

Run baby run

Frida

Inject small JavaScript hooks into a running app to read arguments, change return values, or defeat a check live.

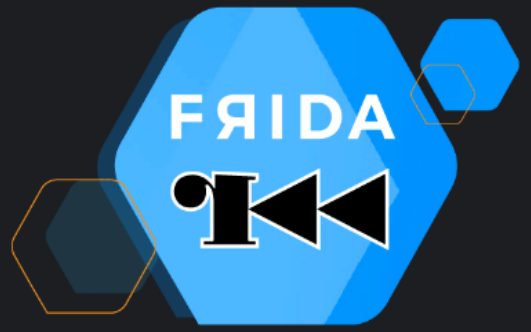
objection

A Frida-powered toolkit with ready-made commands: disable SSL pinning or root detection in one line.

Burp Suite

Sit between the app and its server to read and tamper with API traffic.





Checklist

Open jadx, run MobSF, and skim for these first. These are the easiest vuln compared to other:

- Hardcoded secrets: keys, passwords, tokens in code, res/, or assets/
- Exported components: activities, services, broadcast receivers, and Content Providers other apps can reach
- Client-side checks: root / emulator detection the app trusts and you can hook
- Insecure storage: SharedPreferences, SQLite, files, and logs holding sensitive data
- Cleartext traffic and misconfigured network security

LAB

AndroGoat

AndroGoat is a deliberately vulnerable Android app (Kotlin, 33 challenges, even cited in OWASP MASTG)

Today we'll do two of its challenges live:

- **Root Detection**
- **Network Intercepting**

ROOTING EMULATOR

Android Studio is needed

And we will use rootAVD to help us and rooting

You can found in here

https://developer.android.com/studio

https://gitlab.com/newbit/rootAVD





Go Go Go

Have You learned Enough?

The takeaways

- Everything that works client side, possibility for finding vulnerability is high.

Now you try

- More AndroGoat
- Frida codeshare + OWASP MASTG
- Try on Mobile Hacking Lab, (ps: hacking lab is free and free course)



QnA Session